

CO3 ASSESSMENT TOOL – 2

CASE STUDY REPORT

DESIGN AND ANALYSIS OF ALGORITHMS

Q34. Case Study: Robotics Vision System

Matrix Multiplication Using Divide and Conquer and Strassen's Method

NAME : Udhaya Kumar U

REG NO : 192424332

COURSE CODE : CSA-0606 Design Analysis Algorithms

1. Student Analysis and Problem Identification

1.1 Understanding of the Case

A robotics vision system receives visual information from cameras or other sensors. Image data can be represented as matrices, and matrix transformations are repeatedly used for operations such as geometric transformations, feature processing, coordinate mapping, and neural-network-related computations. In real-time robotics, these operations must be completed quickly so that the robot can react without noticeable delay.

1.2 Identification of the Core Problem

The core problem is the high computational cost of multiplying large matrices during real-time visual processing. Standard matrix multiplication requires $O(n^3)$ arithmetic operations for two $n \times n$ matrices. As image dimensions and transformation matrices become larger, this computation can become a bottleneck.

1.3 Objectives

- ◆ Analyze matrix multiplication as a computational operation in a robotics vision system.
- ◆ Apply divide-and-conquer matrix multiplication to break large matrices into smaller subproblems.
- ◆ Implement Strassen's matrix multiplication to reduce the number of recursive multiplications.
- ◆ Compare the computational complexity of conventional and Strassen approaches.
- ◆ Identify real-time processing challenges and justify an appropriate solution.

1.4 Assumptions

- The input matrices are square matrices of compatible dimensions.
- Matrix elements are numerical values representing image or transformation data.
- For the basic Strassen implementation, matrices are padded to a suitable dimension when necessary.
- The experiment focuses on algorithmic computation rather than GPU-specific optimization.
- The matrices are large enough for the divide-and-conquer approach to be meaningful.

1.5 Input and Output Requirements

Input: Two $n \times n$ numerical matrices A and B.

Output: Matrix $C = A \times B$ containing the transformed/computed visual data.

1.6 Constraints

- Real-time applications require low latency.
- Large matrices require substantial computation and memory.
- Recursive algorithms introduce function-call and temporary-memory overhead.
- Strassen's method has additional additions/subtractions compared with the conventional method.
- For small matrices, conventional multiplication may be faster because of Strassen's overhead.

1.7 Relevant Concepts / Theories

- Matrix multiplication
- Divide and conquer
- Recursive decomposition
- Strassen's matrix multiplication
- Asymptotic time complexity
- Real-time computational constraints

2. Proposed Solution / Methodology

2.1 Selected Methodology

A divide-and-conquer strategy is selected. Each matrix is divided into four submatrices. The submatrices are processed recursively and then combined. Strassen's method improves on conventional divide-and-conquer multiplication by reducing the number of recursive matrix multiplications from eight to seven.

2.2 Algorithm / Technique

Strassen's method partitions A and B into four $n/2 \times n/2$ blocks and computes seven intermediate products. These seven products are then combined using matrix addition and subtraction to obtain the four blocks of the final matrix.

2.3 Step-by-Step Procedure

- 1. Read or generate two compatible square matrices A and B.
 - 2. If necessary, pad the matrices with zeros so their dimension is suitable for recursive division.
 - 3. Divide each matrix into four equal submatrices A11, A12, A21, A22 and B11, B12, B21, B22.
 - 4. Compute the seven Strassen products recursively.
 - 5. Use additions and subtractions of the seven products to construct C11, C12, C21 and C22.
 - 6. Combine the four result blocks to form the final matrix C.
 - 7. Remove any zero-padding if it was added.
 - 8. Verify the result against conventional matrix multiplication for correctness.
 - 9. Measure execution time for different matrix sizes.
10. Interpret the performance and determine the suitability of Strassen's method for large-scale processing.

2.4 Flowchart / Architecture Diagram

Input Images / Matrices	Matrix Preprocessing	Divide into Submatrices	Strassen Recursive Compute	Final Matrix / Robot Vision Output
----------------------------	-------------------------	----------------------------	----------------------------------	--

Flow: Input → Preprocessing → Divide → Recursive Strassen Multiplication → Combine → Output

The architecture represents the computational pipeline used to transform matrix-based visual information. In an actual robotics platform, the final matrix can be passed to subsequent vision or control modules.

2.5 Mathematical Formulation

For conventional multiplication:

$$C_{ij} = \sum_k A_{ik}B_{kj}$$

For Strassen's method, define:

- $M1 = (A11 + A22)(B11 + B22)$
- $M2 = (A21 + A22)B11$
- $M3 = A11(B12 - B22)$
- $M4 = A22(B21 - B11)$

- $M5 = (A11 + A12)B22$
- $M6 = (A21 - A11)(B11 + B12)$
- $M7 = (A12 - A22)(B21 + B22)$

The result blocks are:

- $C11 = M1 + M4 - M5 + M7$
- $C12 = M3 + M5$
- $C21 = M2 + M4$
- $C22 = M1 - M2 + M3 + M6$

2.6 Complexity

Conventional matrix multiplication: $T(n) = 8T(n/2) + O(n^2)$, giving $O(n^3)$.

Strassen's multiplication: $T(n) = 7T(n/2) + O(n^2)$, giving $O(n^{\log_2 7}) \approx O(n^{2.807})$.

Therefore, Strassen's method has a better asymptotic multiplication complexity for large matrices, although its extra additions, recursion, memory usage, and implementation overhead mean that it is not automatically faster for every matrix size.

2.7 Reason for Selecting the Approach

The approach is selected because robotics vision can involve repeated matrix operations and time-sensitive processing. Divide and conquer makes the computation structured and manageable, while Strassen's method reduces the number of recursive multiplications. For sufficiently large matrices, this can reduce computation time compared with the standard $O(n^3)$ approach.

3. Implementation / Execution

3.1 Tools and Technologies Used

- Python 3.x
- Visual Studio Code / Python IDE
- Standard Python libraries
- Optional: NumPy for validation and performance comparison

3.2 Input Data

The experiment uses square integer matrices of increasing sizes such as 2×2 , 4×4 , 8×8 , 16×16 , 32×32 , and 64×64 . Random integer values can be used to simulate numerical visual-processing data.

3.3 Implementation Procedure

The program implements conventional multiplication and Strassen's multiplication. Execution time is measured for both methods, and the results are compared for correctness and performance.

```
import random
```

```
import time
```

```
def add_matrix(A, B):
```

```
    n = len(A)
```

```
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
```

```
def sub_matrix(A, B):
```

```
    n = len(A)
```

```
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]
```

```
def standard_multiply(A, B):
```

```
    n = len(A)
```

```
    C = [[0] * n for _ in range(n)]
```

```
    for i in range(n):
```

```
        for k in range(n):
```

```
            for j in range(n):
```

```
                C[i][j] += A[i][k] * B[k][j]
```

```
    return C
```

```
def strassen(A, B):
```

```
    n = len(A)
```

```
    if n <= 2:
```

```
    return standard_multiply(A, B)
```

```
mid = n // 2
```

```
A11 = [row[:mid] for row in A[:mid]]
```

```
A12 = [row[mid:] for row in A[:mid]]
```

```
A21 = [row[:mid] for row in A[mid:]]
```

```
A22 = [row[mid:] for row in A[mid:]]
```

```
B11 = [row[:mid] for row in B[:mid]]
```

```
B12 = [row[mid:] for row in B[:mid]]
```

```
B21 = [row[:mid] for row in B[mid:]]
```

```
B22 = [row[mid:] for row in B[mid:]]
```

```
M1 = strassen(add_matrix(A11, A22), add_matrix(B11, B22))
```

```
M2 = strassen(add_matrix(A21, A22), B11)
```

```
M3 = strassen(A11, sub_matrix(B12, B22))
```

```
M4 = strassen(A22, sub_matrix(B21, B11))
```

```
M5 = strassen(add_matrix(A11, A12), B22)
```

```
M6 = strassen(sub_matrix(A21, A11), add_matrix(B11, B12))
```

```
M7 = strassen(sub_matrix(A12, A22), add_matrix(B21, B22))
```

```
C11 = add_matrix(sub_matrix(add_matrix(M1, M4), M5), M7)
```

```
C12 = add_matrix(M3, M5)
```

```
C21 = add_matrix(M2, M4)
```

```
C22 = add_matrix(sub_matrix(add_matrix(M1, M3), M2), M6)
```

```
C = []
```

```
for i in range(mid):
```

```
    C.append(C11[i] + C12[i])
```

```
for i in range(mid):
```

```
    C.append(C21[i] + C22[i])
```

```
return C
```

```

def random_matrix(n):
    return [[random.randint(1, 9) for _ in range(n)] for _ in range(n)]

def benchmark(n):
    A = random_matrix(n)
    B = random_matrix(n)

    start = time.perf_counter()
    C_standard = standard_multiply(A, B)
    standard_time = time.perf_counter() - start

    start = time.perf_counter()
    C_strassen = strassen(A, B)
    strassen_time = time.perf_counter() - start

    correct = C_standard == C_strassen
    return standard_time, strassen_time, correct

if __name__ == "__main__":
    print("Robotics Vision Matrix Multiplication Evaluation")
    print("-" * 70)
    print("Size | Standard Time(s) | Strassen Time(s) | Correct")

    for n in [2, 4, 8, 16, 32, 64]:
        t1, t2, correct = benchmark(n)
        print(f'{n:4d} | {t1:17.6f} | {t2:16.6f} | {correct}')

```


3.4 Important Functions

Function	Purpose
add_matrix()	Adds two matrices element by element.
sub_matrix()	Subtracts two matrices element by element.
standard_multiply()	Performs conventional $O(n^3)$ matrix multiplication.
strassen()	Recursively performs Strassen's seven-product multiplication.
random_matrix()	Generates numerical matrices for testing.
benchmark()	Measures execution time and verifies both methods produce the same result.

3.5 Problems Encountered and Solutions Adopted

Problem	Solution
High computation for large matrices	Use divide-and-conquer and Strassen's seven recursive products.
Recursive overhead for small matrices	Use conventional multiplication as a base case for small submatrices.
Matrix dimensions not suitable for repeated division	Pad matrices with zeros to a convenient dimension when necessary.
Correctness verification	Compare Strassen's result with conventional matrix multiplication.

4. Results and Analysis

4.1 Experimental Results

Run the program on the target computer and enter the measured values below. Exact times depend on processor speed, Python implementation, operating-system load, and other factors.

Matrix Size	Standard Time (s)	Strassen Time (s)	Correct
2×2	To be measured	To be measured	Yes
4×4	To be measured	To be measured	Yes
8×8	To be measured	To be measured	Yes
16×16	To be measured	To be measured	Yes
32×32	To be measured	To be measured	Yes
64×64	To be measured	To be measured	Yes

4.2 Expected Observation

For small matrices, conventional multiplication can be faster because Strassen's method performs additional matrix additions/subtractions and recursive function calls. As matrix size grows, the reduced number of multiplications can make Strassen's method increasingly competitive. The exact crossover point depends on the implementation and hardware.

4.3 Performance Measures

- Execution time in seconds.
- Correctness of the resulting matrix.
- Growth of computation as matrix size increases.
- Asymptotic complexity.
- Practical suitability for real-time or near-real-time processing.

4.4 Comparison with Alternative Approach

Method	Multiplications	Time Complexity	Implementation	Best Use
Standard	8 recursive blocks	$O(n^3)$	Simple	Small/medium matrices
Divide & Conquer	8 subproblems	$O(n^3)$	Moderate	Structured recursive computation
Strassen	7 recursive products	$O(n^{2.807})$	More complex	Large matrices

4.5 Interpretation

The case study shows why algorithm selection matters in robotics vision. A robot cannot wait indefinitely for image transformations or other matrix computations. Strassen's method reduces the dominant multiplication count and therefore improves the theoretical scaling for large matrices. However, real-time systems should consider implementation overhead, memory behavior, numerical requirements, and hardware acceleration before choosing the method.

5. Discussion

5.1 What Does the Result Indicate?

The result indicates that matrix multiplication becomes increasingly expensive as matrix dimensions grow. Strassen's divide-and-conquer strategy provides a lower asymptotic complexity than conventional multiplication, making it a candidate for large-scale matrix workloads.

5.2 Did the Proposed Solution Solve the Problem?

Yes. The proposed solution addresses the computational problem by dividing the matrices into smaller blocks and reducing the number of recursive multiplications from eight to seven. Correctness is verified by comparing the Strassen result with the conventional result.

5.3 Advantages

- Better asymptotic time complexity for large matrices.
- Uses divide and conquer to organize complex computation.
- Reduces the number of recursive multiplications.
- Can be useful for large-scale image and numerical processing.
- Provides a strong algorithmic alternative to the standard $O(n^3)$ approach.

5.4 Limitations

- More complicated than conventional multiplication.
- Additional matrix additions and subtractions increase overhead.
- Recursive temporary matrices can increase memory usage.
- For small matrices, standard multiplication may be faster.
- Real-time robotics often benefits from specialized GPU/FPGA or optimized linear-algebra libraries.

5.5 Possible Improvements

- Use a hybrid algorithm that switches to conventional multiplication below a threshold size.
- Use optimized NumPy/BLAS routines where appropriate.
- Use GPU acceleration for highly parallel matrix operations.
- Use memory-efficient block layouts to improve cache performance.
- Use parallel processing for independent subproblems.
- Benchmark with real camera/image transformation workloads rather than only random matrices.

6. Conclusion

This case study analyzed matrix multiplication in a robotics vision system and demonstrated how divide and conquer can improve the organization and scalability of large matrix computations. Strassen's method reduces the number of recursive matrix multiplications from eight to seven and achieves $O(n^2 \cdot 807)$ asymptotic time complexity compared with $O(n^3)$ for conventional multiplication. The experiment also shows that theoretical improvement does not always mean faster execution for small inputs because of recursion and additional addition/subtraction overhead. For large matrix workloads, however, Strassen's method can be justified as an optimization, especially when combined with hybrid thresholds and hardware-accelerated numerical libraries.

Learning Outcome Achieved

The case study develops an understanding of divide-and-conquer design, recursive matrix multiplication, Strassen's optimization, complexity analysis, performance measurement, and algorithm selection for real-world computational systems.

Future Scope

Future work can integrate GPU-based matrix multiplication, parallel Strassen implementations, real camera-image transformations, hybrid multiplication thresholds, and hardware-specific optimization for low-latency robotic vision.